0/77 topics

Math

4 min read | Updated 16 Jul 2026 | Topic 70 of 77 | Bookmark | Mark done

23N. Math & Number Theory (8 problems)

Pattern recognition: GCD/LCM, primes, exponentiation, precision, number properties.

23.168 GCD / LCM

JAVASCRIPT Copy

```javascript
function gcd(a, b) { return b === 0 ? a : gcd(b, a % b); }
function lcm(a, b) { return (a / gcd(a, b)) * b; }
```

23.169 Sieve of Eratosthenes

JAVASCRIPT Copy

```javascript
function sieve(n) {
  const isPrime = Array(n + 1).fill(true);
  isPrime[0] = isPrime[1] = false;
  for (let i = 2; i * i ≤ n; i++) {
    if (isPrime[i]) {
      for (let j = i * i; j ≤ n; j += i) isPrime[j] = false;
    }
  }
```

23.170 Power Function (Binary Exponentiation)

JAVASCRIPT Copy

```
function myPow(x, n) {
  if (n < 0) { x = 1 / x; n = -n; }
  let result = 1;
  while (n > 0) {
    if (n & 1) result *= x;
    x *= x;
    n >>= 1;
  }
  return result;
}
```

23.171 Floating Point Precision in JavaScript

Approach: Because doubles cannot represent 0.1/0.2 exactly, avoid strict === on decimals and instead compare within Number.EPSILON tolerance or round to fixed places.

JAVASCRIPT Copy

```
// Numbers are IEEE-754 doubles, so decimals like 0.1 aren't exact.
console.log(0.1 + 0.2);            // 0.30000000000000004
console.log(0.1 + 0.2 === 0.3);    // false  ⟵ the classic trap

// Safe comparison 1: tolerance using Number.EPSILON
function nearlyEqual(a, b, eps = Number.EPSILON) {
  return Math.abs(a - b) < eps;
}
console.log(nearlyEqual(0.1 + 0.2, 0.3)); // true
```

```
// Round to a fixed number of decimals
console.log(Number((0.1 + 0.2).toFixed(2)) === 0.3); // true
```

`0.1 + 0.2 !== 0.3`. Fix: compare with `Math.abs(a - b) < Number.EPSILON`, or use integer arithmetic (store cents not dollars).

23.172 Happy Number

Approach: Repeatedly replace n with the sum of the squares of its digits, using a Set to detect a cycle; n is happy iff it reaches 1.

JAVASCRIPT Copy

```javascript
function isHappy(n) {
  const seen = new Set();
  while (n !== 1 && !seen.has(n)) {
    seen.add(n);
    let sum = 0;
    while (n > 0) {
      const d = n % 10;
      sum += d * d;
      n = Math.floor(n / 10);
    }
    n = sum;
  }
  return n === 1;
}
```

Sum of squares of digits, detect cycle with Set or Floyd's algorithm.

23.173 Roman to Integer

Approach: Scan left to right; subtract a symbol's value when it is smaller than the one after it (subtractive pairs), otherwise add it.

```javascript
function romanToInt(s) {
  const map = { I: 1, V: 5, X: 10, L: 50, C: 100, D: 500, M: 1000 };
  let total = 0;
  for (let i = 0; i < s.length; i++) {
    const cur = map[s[i]];
    const next = map[s[i + 1]] || 0;
    total += cur < next ? -cur : cur;
  }
  return total;
}
```

Map values, handle subtractive cases (IV=4, IX=9). If current < next, subtract; else add.

23.174 Integer to Roman

Approach: Greedily subtract the largest possible value from a descending table of values (including subtractive forms like 900/400/90/40/9/4), appending its symbol each time.

JAVASCRIPT Copy

```javascript
function intToRoman(num) {
  const vals = [1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1];
  const syms = ['M', 'CM', 'D', 'CD', 'C', 'XC', 'L', 'XL', 'X', 'IX', 'V'
  let res = '';
  for (let i = 0; i < vals.length; i++) {
    while (num ≥ vals[i]) {
      res += syms[i];
      num -= vals[i];
    }
  }
  return res;
}
```

Greedy: subtract largest value first. Use arrays of `[value, symbol]` pairs sorted descending.

23.175 Factorial Trailing Zeros

Approach: Trailing zeros equal the number of factors of 5 in n!, so sum floor(n/5) + floor(n/25) + floor(n/125) + ...

JAVASCRIPT Copy

```javascript
function trailingZeroes(n) {
  let count = 0;
  for (let p = 5; p ≤ n; p *= 5) {
    count += Math.floor(n / p);
  }
  return count;
}
```

Count factors of 5 in n!: `n/5 + n/25 + n/125 + ...`

QUICK CHECK — no marks, just for you.

1. The lesson's Euclidean gcd and the lcm built on top of it are called below. What is printed?

JAVASCRIPT Copy

```javascript
function gcd(a, b) { return b === 0 ? a : gcd(b, a % b); }
function lcm(a, b) { return (a / gcd(a, b)) * b; }

console.log(gcd(48, 18));
console.log(lcm(4, 6));
```

- ○ 6
12
- ○ 2
12

- 12
  6

Check answer

2. This demonstrates the floating point issue the lesson describes, and the suggested fix. What is printed?

JAVASCRIPT Copy

```
console.log(0.1 + 0.2 === 0.3);
console.log(Math.abs((0.1 + 0.2) - 0.3) < Number.EPSILON);
```

- false
  true
- true
  true
- false
  false
- true
  false

Check answer

3. Using the counting rule the lesson gives for factorial trailing zeros, how many trailing zeros does 30! have?

- 8
- 7
- 6

Check answer

Previous
Bit manipulation

Next
Miscellaneous

LEARN MORE

ARTICLES & DOCS

Mathematical algorithms (GfG)
geeksforgeeks.org

VIDEOS



Math & number theory for interviews
YouTube



Top maths questions for interviews
YouTube



Number theory for coding interviews
YouTube

Hand-picked references for this lesson.